

Engineering Intern — Take-Home

Build a minimal AI exam grader, end to end

Zanista

Contact	careers@zanista.ai
Deadline	5 calendar days from receipt

The task

Build a minimal AI exam grader.

Input: an exam paper PDF (questions plus the official mark scheme) and a handwritten student-answer PDF. Output: a structured grading result — per-question scores and feedback — produced by gpt-5.4-mini.

The grading is the heart of the project. Auth, storage, Docker, and everything else is plumbing around one job: turning a stack of handwritten student answers into a clean, per-question score a teacher can trust.

Inputs

- **Exam PDF** — multi-page, English. Questions and official mark scheme.
- **Student answers PDF** — multi-page, English, real handwriting.
- **Azure OpenAI access** — we send an API key, an endpoint URL, an API version, and a deployment name (the deployment is configured to use gpt-5.4-mini). Use the **Azure OpenAI** client for your stack, not the public OpenAI client. Treat \$20 as your usage budget — we monitor resource usage and will revoke the key if you exceed it. See `API_QUICKSTART.pdf` in the bundle for ready-to-use snippets.

Sample PDFs ship with this assignment. Your app must work for any similar pair. Expect 3–15 pages per PDF and 5–15 questions per exam. Inside the exam PDF, the questions and mark scheme may be interleaved, sequential, or split across labelled sections — do not hard-code one layout.

Output (what “graded properly” means)

Every grading run produces a structured result that includes, at a minimum:

- A score for each question — awarded marks and max marks.
- A total score across all questions.
- Feedback per question explaining what was right or wrong.

Machine-readable (JSON or any structured format). Complete: every question in the exam appears in the output, including ones the student left blank.

You design the schema. Field names, nesting, feedback shape — all yours. Pick something you can defend.

We grade structure, consistency, and completeness — **not** absolute marking accuracy. Prompt design counts; model correctness on every mark does not.

Required — automatic rejection if any of these fail

These are the non-negotiable items. Missing any one of them ends the review immediately. No partial credit.

- **Runs from a clean clone with docker compose up.** No manual steps, no missing dependencies.
- **Backend HTTP API.** Any language, any framework. Endpoints to register / log in / submit a grading / list past gradings / retrieve a grading.
- **Real authentication.** The backend rejects unauthenticated requests. Passwords are hashed, never stored in plaintext. Auth cannot be bypassed by editing client-side state.
- **Persistence in a database of your choice.** SQLite, Postgres, MongoDB, or any other store — whatever you want, as long as it survives container restarts and is scoped per user. Every past grading must be loadable and viewable later by the user who produced it.
- **Hard request limit: 120 seconds.** A grading endpoint returns within 2 minutes or responds with HTTP 504. Real cancellation, not a `timeout=` parameter. Synchronous HTTP is fine; if you use async jobs with polling, the same 120-second budget applies from submission to a retrievable result.
- **Model.** Use only the deployment we provision — `gpt-5.4-mini`. No fallback to other deployments on the resource and no other models. The same API key technically reaches other deployments; calling any of them is an automatic rejection and we audit usage.
- **Azure OpenAI credentials from environment variables.** API key, endpoint URL, deployment name, and API version. Never committed, never baked into the image.
- **No secrets, .env files, or database files committed** to the repo.
- **README at the root** with all project documentation: how to run, where to set the Azure OpenAI credentials, the API endpoints (especially if you did not ship a UI), the database setup, design decisions, known limitations, and a short note on how you used AI tools.

Open choices

Language, framework, database, auth library, PDF handling (text vs. images vs. both), prompt design, output schema, number of LLM calls per grading. Pick what you can defend on the follow-up call.

Bonus — optional, but everything here is read carefully

Not required. One or two well-executed bonuses beat a long list of half-done. These are the strongest positive signals we look for.

- **Frontend.** A web UI for upload, results, and past gradings. Strong bonus on its own.
- **Clean code principles.** Readable, well-named, organised. A reviewer who has not used your stack should be able to navigate the project without help. Small functions, sensible module boundaries, no dead code.
- **Proper logging with per-run cost calculation.** Each grading writes a structured log line containing the deployment, input tokens, output tokens, and the **exact USD cost** (computed from Azure

OpenAI's published per-token rates for gpt-5.4-mini). Errors and LLM-call durations are logged at the appropriate level. Visible by tailing the container's stdout.

- **Full grading flow under 5 minutes end-to-end** — upload to result visible.
- **Per-mark traceability.** Each mark in the output cites the page or section of the student's answer that produced it.
- **Teacher override.** The user can change a mark, save the override, and see it persist on next view.
- **Multi-student support.** One exam plus several student PDFs in a single run.

Submission

Submit a ZIP of your project folder. Layout:

```
project/
├── backend/
├── frontend/           (only if you built one)
├── docker-compose.yml
└── README.md
```

Email the ZIP to **careers@zanista.ai**.

After we review the code, we will schedule a **20-minute follow-up call**. Have your repo open. We will walk through auth, the time limit, your prompt, the output schema, and how you used AI tools.

You are welcome to keep the project on your GitHub afterwards.

Rules

The work must be your own. AI tools, libraries, and templates are fine — cite anything substantial in your README. Do not redistribute this assignment, the sample PDFs, or the Azure OpenAI credentials. By submitting, you confirm the code is yours and that we may run, read, and discuss it as part of evaluation.

If anything is unclear, email us using the address above. Good luck.